

64. WebSocket

WebSocket protocol (**RFC 6455**) browser aur server ke beech **persistent connection** deta hai. Data dono taraf se "packets" me jaata hai, bina connection toote aur bina extra HTTP requests ke.

Bahut achha hai un services ke liye jinhe lagatar data exchange chahiye: online games, real-time trading systems, chat.

Simple example

```
js
let socket = new WebSocket("ws://javascript.info");
```

`ws` protocol use hota hai. Encrypted version `wss://` hai (jaise HTTPS).

Hamesha `wss://` prefer karo. Ye sirf encrypted hi nahi, zyaada **reliable** bhi hai: `ws://` ka data encrypted nahi hota (beech ke intermediary dekh sakte hain), aur purane proxies WebSocket ko nahi jaante, "ajeeb" headers dekh ke connection tod sakte hain. `wss://` TLS par WebSocket hai, data encrypted guzarta hai, proxies andar dekh nahi paate aur guzarne dete hain.

4 events

- `open`: connection ban gaya
- `message`: data mila
- `error`: websocket error
- `close`: connection band

Bhejna: `socket.send(data)`.

```
js
```

```

let socket = new WebSocket("wss://javascript.info/article/websocket/demo/hello")

socket.onopen = function(e) {
  alert("[open] Connection established");
  socket.send("My name is John");
};

socket.onmessage = function(event) {
  alert(`[message] Data received from server: ${event.data}`);
};

socket.onclose = function(event) {
  if (event.wasClean) {
    alert(`[close] Connection closed cleanly, code=${event.code} reason=${event.reason}`);
  } else {
    alert('[close] Connection died'); // aam taur par code 1006
  }
};

socket.onerror = function(error) { alert(`[error]`); };

```

Websocket kholna (handshake)

`new WebSocket(url)` banate hi connect hone lagta hai. Browser headers se server se puchhta hai: "Kya tum WebSocket support karte ho?" Server "haan" kahe to aage baat **WebSocket protocol** me hoti hai (jo HTTP nahi hai).

Request headers (jaise `wss://javascript.info/chat` ke liye):

```

GET /chat
Host: javascript.info
Origin: https://javascript.info
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Key: Iv8io/9s+lYFgZWcXczP8Q==
Sec-WebSocket-Version: 13

```

- `Origin`: client page ka origin. WebSocket **cross-origin by nature** hai (koi special headers/limitations nahi), par `Origin` se server decide karta hai ki is site se baat karni hai ya nahi.
- `Connection: Upgrade`: protocol badalna chahta hai
- `Upgrade: websocket`: chahiya hua protocol

- `Sec-WebSocket-Key`: random key, server ke WebSocket support ki jaanch ke liye (random taaki proxies aage ki baat cache na karein)
- `Sec-WebSocket-Version`: 13 (current)

Ye handshake `XMLHttpRequest` ya `fetch` se emulate nahi ho sakta, kyunki JS ye headers set nahi kar sakti.

Server agree kare to **101** response:

```
101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: hsB1buDTkk24srzE0TBu1ZA1C2g=
```

`Sec-WebSocket-Accept` = `Sec-WebSocket-Key` ko special algorithm se recode kiya hua. Isse browser ko pata chalta hai ki server sach me WebSocket samajhta hai.

Extensions aur subprotocols

- `Sec-WebSocket-Extensions: deflate-frame`: compression jaisi cheez (browser khud bhejta hai)
- `Sec-WebSocket-Protocol: soap, wamp`: data ka format (SOAP, WAMP). Ye `new WebSocket` ke **doosre parameter** se jaata hai:

```
js
let socket = new WebSocket("wss://javascript.info/chat", ["soap", "wamp"]);
```

Server jin ko manta hai unki list wapas bhejta hai.

Data transfer

Communication "**frames**" (data ke tukde) me hoti hai:

- text frames
- binary frames
- ping/pong frames (server connection check karta hai, browser khud jawab deta hai)
- close frame aur kuch service frames

Browser me hum sirf text/binary frames se seedha kaam karte hain.

`socket.send(body)` string ya binary (`Blob`, `ArrayBuffer`...) dono bhej sakta hai.

Data lene par: text hamesha **string** milta hai. Binary ke liye `Blob` ya `ArrayBuffer` chun sakte ho, `socket.binaryType` se (default `"blob"`):

```
js

socket.binaryType = "arraybuffer";
socket.onmessage = (event) => {
  // event.data string (text) ya arraybuffer (binary)
};
```

Rate limiting

Bahut data bhejna ho aur user ka network slow ho, to `send` baar baar call karne par data memory me **buffer** hota rehta hai. `socket.bufferedAmount` batata hai ki abhi kitne bytes bhejne baaki hain.

```
js

setInterval(() => {
  if (socket.bufferedAmount == 0) {
    socket.send(moreData());
  }
}, 100);
```

Connection band karna

Dono taraf ko barabar haq hai. Band karne wala "close frame" bhejta hai (numeric code + text reason):

```
js

socket.close([code], [reason]);

socket.close(1000, "Work complete"); // closing party
socket.onclose = event => {
  // event.code === 1000, event.reason === "Work complete", event.wasClean ===
};
```

Common codes:

- `1000`: normal closure (default)
- `1006`: connection kho gaya (close frame nahi mila). **Ye manually set nahi kar sakte.**
- `1001`: party ja rahi hai (server shutdown, browser page chhod raha)
- `1009`: message bahut bada
- `1011`: server par unexpected error

1000 se chhote codes reserved hain (set karne par error). Toote connection par `code = 1006`, `reason = ""`, `wasClean = false`.

Connection state: `socket.readyState`

- `0` CONNECTING: abhi nahi bana
- `1` OPEN: baat ho rahi hai
- `2` CLOSING: band ho raha hai
- `3` CLOSED: band

Chat example

HTML: message ke liye `<form>` aur incoming messages ke liye `<div>`.

```
js

let socket = new WebSocket("wss://javascript.info/article/websocket/chat/ws");

document.forms.publish.onsubmit = function() {
  socket.send(this.message.value);
  return false;
};

socket.onmessage = function(event) {
  let messageElem = document.createElement('div');
  messageElem.textContent = event.data;
  document.getElementById('messages').prepend(messageElem);
};
```

Server (Node.js, `ws` module) ka idea:

1. `clients = new Set()`
2. Har naye socket ko `clients.add(socket)` karo aur `message` listener lagao
3. Message aane par saare clients ko bhejo
4. Connection band hone par `clients.delete(socket)`

Summary

- Modern persistent browser-server connection.
- Cross-origin limitations nahi, browsers me achhe se supported.
- Strings aur binary data dono.
- Methods: `send(data)`, `close([code], [reason])`. Events: `open`, `message`, `error`, `close`.
- Reconnection, authentication jaise high-level features **built-in nahi**, isliye libraries ya khud implement karo.

- Existing projects me aksar main HTTP server ke saath alag WebSocket server (subdomain jaise `wss://ws.site.com`) chalte hain, dono ek database share karte hain.